```
            alpha_bar_t_1 = self.alpha_bars[t-1].to(device)
            beta_t = self.betas[t].to(device)
            sigma_t = torch.sqrt(beta_t * (1 - alpha_bar_t_1) / (1 - alpha_bar_t))
            z = torch.randn_like(x_t).to(device)
            rep = sigma_t * z
        else:
            rep = 0

        x_pred = (1 / torch.sqrt(alpha_t)) * (x_t - noise_coef * pred_noise) + rep
        return x_pred

    def generate(self, x_shape, c):
        x_t = torch.randn(x_shape).to(device)

        for t in reversed(range(self.timesteps)):
            x_t = self.reverse_diffusion(x_t, t, c)

        return x_t
```

Begin Training

```
#@title Begin Training

unet = UNet(in_c=1, out_c=1).to(device)
ddpm = DDPM(model=unet).to(device)
optimizer = optim.AdamW(unet.parameters(), lr=1e-3)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=2)
loss_fn = nn.MSELoss()

num_epochs = 10
for epoch in range(num_epochs):
    unet.train()
    epoch_loss = 0.0
    for x_0, c in tqdm(train_loader, total=len(train_loader)):
        x_0 = x_0.to(device)
        c = c.to(device) # Move class labels to GPU
        # randomly choose timestep t when training
        t = torch.randint(0, ddpm.timesteps, (x_0.size(0),), device=device).long()
        # forward diffusion
        x_t, noise = ddpm.forward(x_0, t)
        # predict noise
        pred_noise = unet(x_t, t, c)
        # compute loss
        loss = loss_fn(pred_noise, noise)
        epoch_loss += loss.item()
```

```
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    epoch_loss /= len(train_loader)
    scheduler.step(epoch_loss)
    current_lr = optimizer.param_groups[0]['lr']
    print(f"Epoch [{epoch+1}/{num_epochs}], Average Loss: {epoch_loss:.4f}, Current LR: {current_lr:.4f}")
```

Show hidden output

Inference

Inference

```
#@title Inference
c = 4 # Label

unet.eval()
with torch.no_grad():
    num_samples = 16
    x_shape = (num_samples, 1, 32, 32)
    generated_images = ddpm.generate(x_shape, c)
    # plot
    plt.figure(figsize=(10,10))
    for i in range(4):
        for j in range(4):
            plt.subplot(4,4,i*4+j+1)
            plt.imshow(generated_images[i*4+j].permute(1,2,0).cpu())
    plt.show()
```